SPADE-like - odkrywanie wzorców sekwencyjnych w języku Python

Raport z realizacji projektu z przedmiotu MED

Autor: Mateusz Wójcik
Nr albumu: 298851

1. Opis zadania projektowego

Celem projektu było opracowanie programu służącego do odkrywania częstych wzorców sekwencyjnych metodą SPADE-like. Zgodnie ze specyfikacją przygotowano cztery samodzielne warianty algorytmu: dSPADE, bSPADE, maxLength-dSPADE oraz maxLength-bSPADE. Pierwsza para przeszukuje pełną przestrzeń wzorców, natomiast druga para kończy rozwijanie wzorca po osiągnięciu zadanej wartości parametru maxLength.

Implementacja odpowiada sposobowi przedstawionemu na wykładzie: dane są przekształcane do reprezentacji pionowej, a wsparcia nowych sekwencji wyznaczane są na podstawie tidlist wzorców rodzicielskich. Kandydaci długości n powstają przez łączenie odpowiednich par częstych wzorców długości n-1. Różnica pomiędzy dSPADE i bSPADE dotyczy wyłącznie porządku eksploracji przestrzeni rozwiązań: dSPADE stosuje przeszukiwanie w głąb, a bSPADE przeszukiwanie wszerz.

Program zapisuje wyniki każdego eksperymentu do dwóch plików: OUT, zawierającego wszystkie odkryte wzorce w kolejności ich znalezienia, oraz STAT, zawierającego opis danych, parametry uruchomienia, pomiary czasu i szczegółowe statystyki kandydatów i wzorców. Eksperymenty wykonano na małym zbiorze z materiałów wykładowych oraz na trzech publicznych zbiorach sekwencyjnych w formacie SPMF: BMSWebView1, SIGN i Kosarak 10k.

2. Podstawowe pojęcia i przyjęte założenia

2.1. Sekwencja, element i pozycja

Sekwencja jest uporządkowaną listą elementów, a każdy element jest zbiorem pozycji występujących w tej samej transakcji. Zapis <(d)(b f)(a)> oznacza, że najpierw wystąpiła pozycja d, następnie w jednej późniejszej transakcji wystąpiły razem pozycje b i f, a w jeszcze późniejszej transakcji wystąpiła pozycja a. Kolejność elementów jest istotna, natomiast kolejność pozycji wewnątrz jednego elementu nie ma znaczenia.

Pojęcie	**Znaczenie**
pozycja (item)	pojedynczy symbol lub identyfikator, np. a, b, 10307
element (itemset)	zbiór pozycji występujących jednocześnie, np. (b f)
sekwencja	uporządkowany ciąg elementów, np. <(d)(b f)(a)>
długość wzorca	łączna liczba pozycji z uwzględnieniem powtórzeń
liczba elementów	liczba itemsetów składających się na wzorzec

2.2. Wsparcie i próg minSup

Wsparcie wzorca jest liczbą różnych sekwencji danych, w których wzorzec występuje. Wielokrotne wystąpienie wzorca w tej samej sekwencji zwiększa długość tidlisty, lecz nie zwiększa wsparcia o więcej niż jeden. W projekcie parametr sup jest bezwzględnym progiem minimalnego wsparcia: wzorzec jest uznawany za częsty, jeżeli jego wsparcie jest nie mniejsze niż sup.

Wykorzystano własność antymonotoniczności: wsparcie nadsekwencji nie może być większe niż wsparcie jej podsekwencji. Jeżeli kandydat nie spełnia progu sup, nie jest rozwijany, ponieważ żaden wzorzec zawierający go jako podsekwencję nie może stać się częsty.

2.3. Założenia implementacyjne

- Pozycje należące do jednego elementu są unikalne i zapisywane w porządku kanonicznym, dzięki czemu ten sam itemset ma zawsze jedną reprezentację.
- Tidlista wzorca jest przechowywana jako odwzorowanie CId -> zbiór TId, w których wzorzec może się kończyć.
- Pełne warianty dSPADE i bSPADE nie są implementowane jako wywołania wariantów maxLength z wartością nieskończoną.
- Warianty DFS i BFS powinny zwracać ten sam zbiór par (wzorzec, wsparcie), lecz mogą zapisywać wzorce w innej kolejności.
- Parametr maxLength odnosi się do długości wzorca rozumianej jako liczba pozycji, a nie liczba elementów.

3. Dane wejściowe i wyjściowe

3.1. Format własny

Pierwszy obsługiwany format opisuje jedną transakcję w każdym wierszu:

```
CId TId item1 item2 ...
```

CId identyfikuje sekwencję, TId określa kolejność transakcji w tej sekwencji, a dalsze pola zawierają pozycje należące do transakcji. Separatorem może być spacja lub przecinek. Wiersze zaczynające się od znaku # są traktowane jako komentarze.

Przykład wiersza „1 15 a b c” oznacza, że w sekwencji klienta 1, w transakcji o identyfikatorze czasowym 15, wystąpiły pozycje a, b i c.

3.2. Natywny format SPMF

Drugim obsługiwanym formatem jest standardowy format baz sekwencyjnych SPMF:

```
item ... -1 item ... -1 -2
```

Znacznik -1 kończy jeden element sekwencji, natomiast -2 kończy całą sekwencję. Każdy wiersz pliku SPMF reprezentuje jedną sekwencję. Podczas odczytu program nadaje wierszom kolejne identyfikatory CId, a elementom kolejne identyfikatory TId. Dzięki temu pliki pobrane z repozytorium SPMF mogą być używane bez ręcznej konwersji i bez zmiany ich semantyki.

Przykład: wiersz „1 2 -1 3 -1 -2” jest interpretowany jako sekwencja <(1 2)(3)>.

3.3. Plik OUT

Plik OUT jest plikiem CSV. Każdy odkryty wzorzec jest zapisywany w osobnym wierszu, w kolejności jego odkrycia przez dany wariant algorytmu. Kolejność ta jest częścią wyniku, dlatego może różnić się pomiędzy DFS i BFS.

Pole	Opis
długość wzorca	łączna liczba pozycji we wzorcu
liczba elementów	liczba itemsetów wzorca

Pole	Opis
długość tidlisty	liczba rekordów (CId,TId) w tidliście
wsparcie sup	liczba różnych sekwencji wspierających wzorzec
wzorzec	kanoniczny zapis sekwencji

3.4. Plik STAT

Plik STAT zawiera informacje pozwalające odtworzyć i ocenić eksperyment. Zapisywane są: nazwa i charakterystyka pliku wejściowego, parametry sup i maxLength, czasy odczytu, eksploracji i zapisu, czas całkowity, maksymalne długości kandydatów i wzorców oraz agregaty dotyczące liczby, sum wsparć i sum długości tidlist. Statystyki są podawane globalnie i osobno dla kolejnych długości wzorców.

Nazwy plików OUT i STAT kodują wariant algorytmu, nazwę zbioru, liczbę sekwencji, transakcji i pozycji oraz wartości parametrów. Ułatwia to jednoznaczne przypisanie pliku do eksperymentu.

4. Projekt i implementacja

4.1. Reprezentacja pionowa i tidlisty

Po wczytaniu bazy tworzona jest reprezentacja pionowa. Zamiast wielokrotnie przeglądać wszystkie transakcje, program przechowuje dla każdej pozycji listę miejsc jej wystąpienia. Rekord tidlisty ma postać (CId,TId), gdzie CId wskazuje sekwencję, a TId transakcję, w której kończy się dane wystąpienie wzorca.

Wsparcie wzorca oblicza się jako liczbę różnych wartości CId w jego tidliście. Długość tidlisty może być większa od wsparcia, ponieważ ten sam wzorzec może kończyć się w kilku transakcjach tej samej sekwencji.

4.2. Łączenia I-join i S-join

Tidlisty kandydatów są wyznaczane na podstawie tidlist wzorców rodzicielskich. W implementacji rozróżniono dwa podstawowe typy rozszerzenia:

Operacja	Warunek czasowy	Znaczenie
I-join / I-extension	końcowe TId są równe	nowa pozycja należy do tego samego elementu
S-join / S-extension	TId nowej pozycji jest większe	nowa pozycja tworzy późniejszy element sekwencji

Przykładowo, z tidlist pozycji d i f można utworzyć zarówno wzorzec <(d f)>, wymagający ich współwystąpienia w tej samej transakcji, jak i <(d)(f)>, wymagający wystąpienia f później niż d. Jest to istotne rozróżnienie semantyczne: pierwszy wzorzec opisuje współwystępowanie, a drugi kolejność w czasie.

4.3. Generowanie kandydatów zgodnie z wersją wykładową

Kandydaci długości n są tworzeni z par częstych wzorców długości n-1, które należą do tej samej klasy równoważności, czyli mają wspólny prefiks. Reguły łączenia zależą od tego, czy ostatnia pozycja rodzica należy do ostatniego itemsetu, czy tworzy osobny element. Program stosuje reguły odpowiadające tabeli przedstawionej na wykładzie.

Rodzic 1	Rodzic 2	Przykładowe dzieci
<G(P)(x)>	<G(P)(y)>	<G(P)(x y)>, <G(P)(x)(y)>, <G(P)(y)(x)>
<G(P)(x)>	ten sam wzorzec	<G(P)(x)(x)>
<G(Px)>	<G(Py)>	<G(Pxy)>
<G(Px)>	<G(P)(y)>	<G(Px)(y)>
<G(P)(x)>	<G(Py)>	<G(Py)(x)>

Każdy kandydat jest kanonizowany, dzięki czemu ten sam wzorzec utworzony z różnych par rodziców jest rejestrowany tylko raz. Następnie obliczana jest jego tidlista, wsparcie i statystyki. Kandydat spełniający próg sup zostaje wzorcem częstym i może uczestniczyć w tworzeniu kolejnego poziomu.

4.4. Strategie przeszukiwania

Wariant	Sposób eksploracji	Ograniczenie długości
dSPADE	rekurencyjne przeszukiwanie w głąb (DFS)	brak
bSPADE	kolejka FIFO i przeszukiwanie wszerz (BFS)	brak
maxLength-dSPADE	rekurencyjne DFS	brak rozwijania po osiągnięciu maxLength
maxLength-bSPADE	kolejka FIFO i BFS	brak rozwijania po osiągnięciu maxLength

DFS kończy eksplorację jednej gałęzi przed przejściem do następnej, dlatego przechowuje przede wszystkim bieżącą ścieżkę i klasy potomne. BFS przetwarza poziomami i może jednocześnie przechowywać dużą liczbę wzorców tej samej długości. Obie strategie tworzą ten sam zbiór częstych wzorców, ale różnią się kolejnością odkrywania i profilem wykorzystania pamięci.

4.5. Pseudokod wspólnego mechanizmu

```
wejście: baza sekwencji D, próg sup, opcjonalnie maxLength

1. odczytaj D i utwórz tidlisty wzorców długości 1
2. odrzuć wzorce o wsparciu mniejszym niż sup
3. umieść częste wzorce w klasach równoważności
4. dla każdej właściwej pary wzorców rodzicielskich:
   a) utwórz kandydatów zgodnie z regułami łączenia
   b) wyznacz tidlistę kandydata przez I-join lub S-join
   c) zarejestruj statystyki kandydata
   d) jeżeli support >= sup, zapisz wzorzec i klasę potomną
5. przetwarzaj klasy potomne rekurencyjnie (DFS) lub kolejką (BFS)
6. zakończ, gdy nie istnieją kolejne klasy lub osiągnięto maxLength
```

4.6. Struktura projektu

Plik lub katalog	Rola
spade_like/core.py	model danych, parsery, tidlisty, generowanie kandydatów, cztery warianty algorytmu i statystyki
spade_like/cli.py	obsługa argumentów wiersza poleceń i uruchamianie eksperymentu

Plik lub katalog	Rola
run.py	punkt wejścia programu
run_experiments.py	automatyczne wykonanie pełnej serii eksperymentów i utworzenie podsumowania
tests/test_spade.py	testy poprawności wyników, formatów i ograniczeń
data/	mały zbiór wykładowy oraz trzy publiczne zbiory SPMF
results/	pliki OUT, STAT i experiment_summary.csv

5. Podręcznik użytkownika

5.1. Wymagania i uruchomienie

Do uruchomienia programu wymagany jest Python 3.10 lub nowszy. Algorytm korzysta z biblioteki standardowej; dodatkowe pakiety są potrzebne wyłącznie do testów i wygenerowania raportu. Polecenia należy wykonywać z głównego katalogu projektu.

Ogólna postać polecenia:

```
python run.py ALGORITHM INPUT --sup SUP [--max-length L] [--out-dir DIR]
```

Argument	Znaczenie
ALGORITHM	dSPADE, bSPADE, maxLength-dSPADE albo maxLength-bSPADE
INPUT	ścieżka do pliku w formacie własnym lub SPMF
--sup	bezwzględny próg minimalnego wsparcia
--max-length	maksymalna długość wzorca; wymagane w wariantach maxLength
--out-dir	katalog docelowy plików OUT i STAT

5.2. Przykłady

```
python run.py dSPADE data/small.txt --sup 2 --out-dir results

python run.py bSPADE data/BMSWebView1.txt --sup 1000 --out-dir results

python run.py maxLength-dSPADE data/sign.txt --sup 400 --max-length 3 --out-dir results

python run.py maxLength-bSPADE data/kosarak10k.txt --sup 1000 --max-length 4 --out-dir results
```

5.3. Testy i pełna seria eksperymentów

Testy automatyczne można uruchomić poleceniem:

```
python -m pytest -q
```

Pełną serię eksperymentów, obejmującą wszystkie zbiory, progi sup i wartości maxLength, uruchamia polecenie:

```
python run_experiments.py
```

Skrypt usuwa wcześniejsze wyniki, wykonuje wszystkie konfiguracje i tworzy plik experiment_summary.csv. Dzięki temu tabele i wykresy w raporcie można odtworzyć na podstawie dołączonych danych.

6. Ilustracja działania na małym zbiorze

6.1. Dane przykładowe

Mały zbiór pochodzi z materiałów wykładowych i obejmuje cztery sekwencje klientów. Dla progu sup=2 program odkrywa 18 częstych wzorców. Zbiór jest wystarczająco mały, aby ręcznie prześledzić działanie tidlist i reguł łączenia.

CId	TId	Pozycje
1	10	c d
1	15	a b c
1	20	a b f
1	25	a c d f
2	15	a b f
2	20	e
3	10	a b f
4	10	d g h
4	20	b f
4	25	a g h

6.2. Przykładowe tidlisty

Dla pozycji d tidlista zawiera rekordy (1,10), (1,25) i (4,10). Długość tidlisty wynosi 3, lecz wsparcie jest równe 2, ponieważ rekordy dotyczą dwóch różnych sekwencji: 1 i 4.

Wzorzec	Tidlista	długość tidlisty	sup
<(d)>	(1,10), (1,25), (4,10)	3	2
<(f)>	(1,20), (1,25), (2,15), (3,10), (4,20)	5	4
<(d f)>	(1,25)	1	1
<(d)(f)>	(1,20), (1,25), (4,20)	3	2

Wzorzec <(d f)> ma wsparcie 1, ponieważ d i f występują razem tylko w transakcji klienta 1 o TId=25. Wzorzec <(d)(f)> ma wsparcie 2, ponieważ u klientów 1 i 4 istnieje wystąpienie f późniejsze od d. Przykład pokazuje, dlaczego algorytm musi rozróżniać I-join od S-join.

6.3. Budowanie dłuższego wzorca

Z częstych wzorców <(d)(b)> i <(d)(f)> można utworzyć między innymi <(d)(b f)>. Następnie po połączeniu z odpowiednim wzorcem kończącym się pozycją a powstaje <(d)(b f)(a)>. Tidlista końcowego wzorca zawiera rekordy (1,25) i (4,25), więc wzorzec występuje u dwóch klientów i spełnia próg sup=2.

Interpretacja biznesowa: klient najpierw kupuje d, później jednocześnie b i f, a następnie a.

6.4. Wyniki wariantów DFS i BFS

Algorytm	Liczba wzorców	Liczba kandydatów	Najdłuższy wzorzec	Czas eksploracji [s]
dSPADE	18	51	4	0.000350
bSPADE	18	51	4	0.000257

Oba warianty zwróciły ten sam zbiór 18 wzorców i tę samą długość maksymalną. Różnica czasu na tak małym zbiorze jest pomijalna i zależy od narzutu wykonania. Istotną różnicą pozostaje kolejność zapisu wzorców w pliku OUT.

7. Eksperymenty na publicznych zbiorach SPMF

7.1. Cel i plan eksperymentów

Celem badań było sprawdzenie poprawności jakościowej wszystkich wariantów oraz zbadanie wpływu parametrów sup i maxLength na liczbę kandydatów, liczbę wzorców i czas wykonania. Dla każdego zbioru zastosowano dwie wartości sup, a dla wariantów ograniczonych wartości maxLength równe 2, 3 i 4. Łącznie wykonano 64 uruchomienia.

Wyniki czasowe należy interpretować jako pomiary dla konkretnej implementacji w Pythonie i konkretnego środowiska uruchomieniowego. Każdą konfigurację wykonano jednokrotnie, dlatego porównania czasu mają charakter orientacyjny; najważniejsze są zależności jakościowe i zgodność wyników.

7.2. Charakterystyka zbiorów

Zbiór	Sekwencje	Transakcje	Pozycje	Śr. trans./sekw.	Min	Max	Śr. poz./trans.	sup
small	4	10	8	2.50	1	4	2.70	2, 3
BMSWebView1	59601	149638	497	2.51	1	267	1.00	1000, 2000
SIGN	730	37958	267	52.00	18	94	1.00	400, 600
Kosarak 10k	10000	81407	10094	8.14	1	608	1.00	1000, 2000

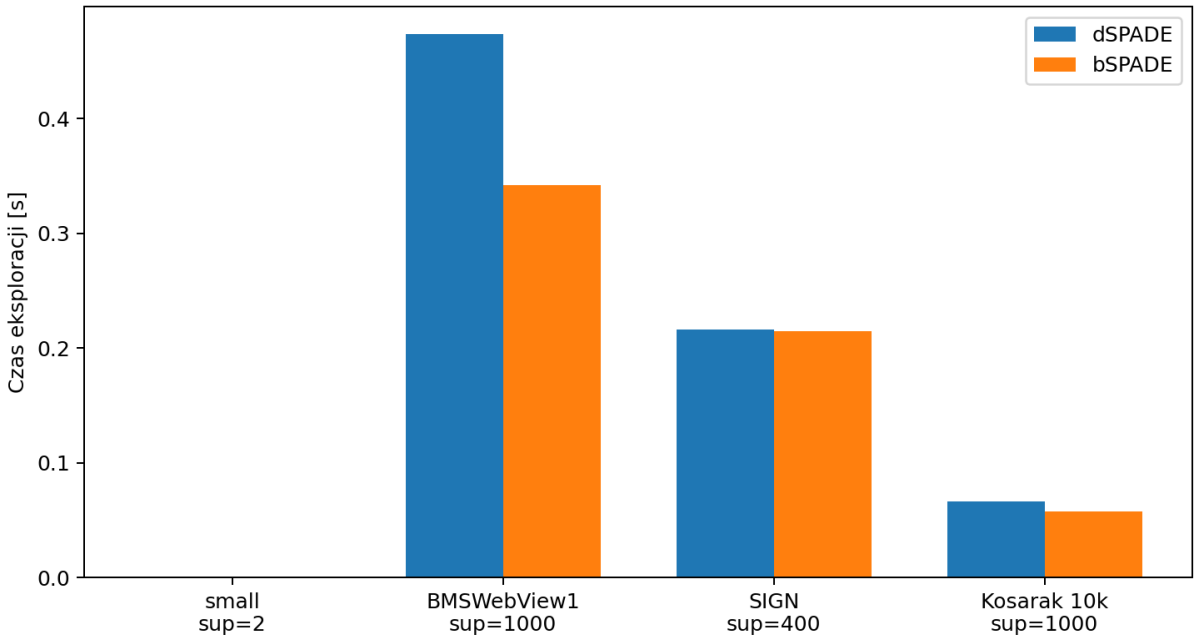
BMSWebView1 zawiera bardzo dużą liczbę stosunkowo krótkich sekwencji i niewielką liczbę różnych pozycji. SIGN ma znacznie mniej sekwencji, ale są one długie, dlatego zbiór dobrze nadaje się do analizy wpływu maxLength. Kosarak 10k ma dużą i rzadką przestrzeń pozycji: liczba różnych identyfikatorów przekracza dziesięć tysięcy. Zestaw trzech baz reprezentuje zatem odmienne profile danych i pozwala sprawdzić zachowanie implementacji w różnych warunkach.

Progi sup dobrano pilotażowo. Dla BMSWebView1 i Kosarak 10k zastosowano 1000 i 2000, a dla SIGN 400 i 600. Wyższe progi są konieczne, aby ograniczyć liczbę kandydatów w implementacji napisanej w czystym Pythonie, a jednocześnie zachować niepuste i możliwe do analizy wyniki.

7.3. Wyniki wariantów pełnych

Zbiór	sup	Algorytm	Wzorce	Kandydaci	Maks. długość	Czas eksploracji [s]
BMSWebView1	1000	dSPADE	31	1833	2	0.474143
BMSWebView1	1000	bSPADE	31	1833	2	0.342213
BMSWebView1	2000	dSPADE	9	614	1	0.178102
BMSWebView1	2000	bSPADE	9	614	1	0.084989
SIGN	400	dSPADE	100	1669	3	0.216144
SIGN	400	bSPADE	100	1669	3	0.214730
SIGN	600	dSPADE	7	319	2	0.017518
SIGN	600	bSPADE	7	319	2	0.017391
Kosarak 10k	1000	dSPADE	9	10124	3	0.066591
Kosarak 10k	1000	bSPADE	9	10124	3	0.057804
Kosarak 10k	2000	dSPADE	5	10108	2	0.046799
Kosarak 10k	2000	bSPADE	5	10108	2	0.046647

Dla każdej pary konfiguracji dSPADE i bSPADE liczba odkrytych wzorców, liczba kandydatów oraz maksymalna długość były takie same. Potwierdza to, że sposób przechodzenia przestrzeni poszukiwań nie wpływa na wynik jakościowy. Różnice czasu nie są jednokierunkowe: w zależności od zbioru i progu nieco szybszy może być DFS lub BFS.

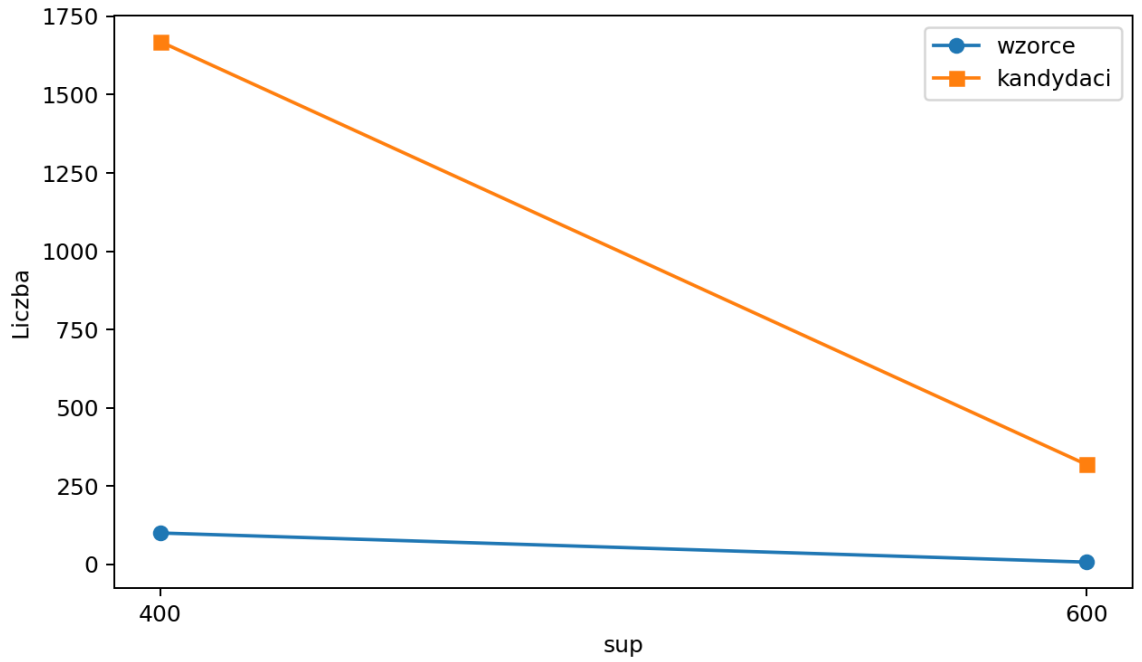


Rys. 1. Porównanie czasu eksploracji dSPADE i bSPADE dla reprezentatywnych progów wsparcia.

7.4. Wpływ progu sup

Zwiększenie progu sup zmniejsza liczbę wzorców uznawanych za częste. W wyniku antymonotoniczności zmniejsza się także liczba wzorców, które mogą zostać rozwinięte na kolejnych poziomach, a więc zwykle również liczba kandydatów i czas eksploracji.

Zbiór	Zmiana sup	Wzorce	Spadek	Kandydaci	Spadek
BMSWebView1	1000 -> 2000	31 -> 9	71.0%	1833 -> 614	66.5%
SIGN	400 -> 600	100 -> 7	93.0%	1669 -> 319	80.9%
Kosarak 10k	1000 -> 2000	9 -> 5	44.4%	10124 -> 10108	0.2%

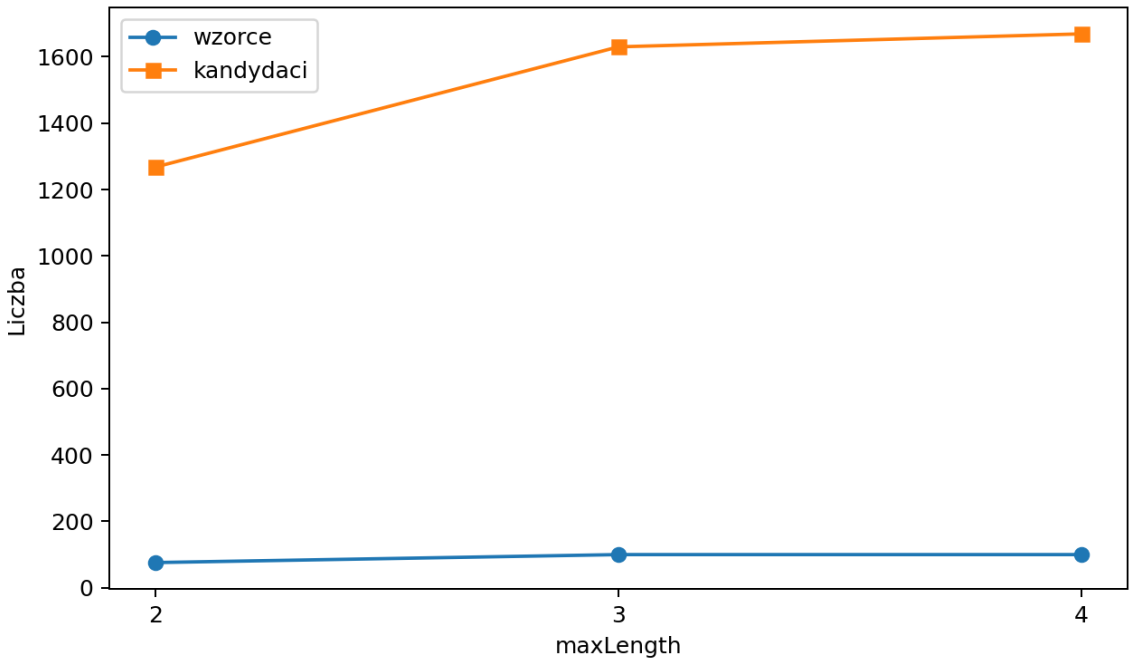


Rys. 2. Wpływ wartości sup na liczbę wzorców i kandydatów dla zbioru SIGN.

7.5. Wpływ parametru maxLength

Parametr maxLength ogranicza długość zwracanych wzorców i zatrzymuje rozwijanie gałęzi po osiągnięciu limitu. Jest szczególnie przydatny w bazach zawierających długie sekwencje, ponieważ zapobiega generowaniu kandydatów, którzy nie są potrzebni użytkownikowi.

Algorytm	maxLength	Wzorce	Kandydaci	Najdłuższy wzorzec	Czas [s]
maxLength-dSPADE	2	76	1268	2	0.156121
maxLength-bSPADE	2	76	1268	2	0.159539
maxLength-dSPADE	3	100	1630	3	0.267094
maxLength-bSPADE	3	100	1630	3	0.227170
maxLength-dSPADE	4	100	1669	3	0.220699
maxLength-bSPADE	4	100	1669	3	0.223287



Rys. 3. Wpływ maxLength na liczbę wzorców i kandydatów dla SIGN, sup=400, wariant DFS.

Dla maxLength=2 program kończy eksplorację na wzorcach dwupozycyjnych. Zwiększenie limitu do 3 i 4 rozszerza przestrzeń poszukiwań i może znacząco zwiększyć liczbę kandydatów. Gdy maxLength jest nie mniejsze od długości najdłuższego częstego wzorca w wariancie pełnym, zbiór wyników wariantu ograniczonego jest identyczny z wynikiem dSPADE lub bSPADE.

7.6. Ocena jakościowa

- Dla każdego zbioru i progu pełne warianty DFS i BFS zwróciły ten sam zbiór par (wzorzec, wsparcie).
- Dla każdej wartości maxLength warianty maxLength-dSPADE i maxLength-bSPADE zwróciły ten sam zbiór wzorców.
- Dla limitu co najmniej równego maksymalnej długości częstego wzorca wynik wariantu ograniczonego był zgodny z wariantem pełnym.
- Na małym zbiorze uzyskano dokładnie 18 wzorców dla sup=2, w tym wzorzec wykładowy <(d)(b f)(a)> ze wsparciem 2.

7.7. Odtwarzalność wyników

Do projektu dołączono wszystkie wykorzystane pliki wejściowe, pliki OUT i STAT oraz zbiorczy plik experiment_summary.csv. Skrypt run_experiments.py wykonuje serię od początku i nadaje wynikom jednoznaczne nazwy. W raporcie wykorzystano bezpośrednio wartości z pliku podsumowującego, dzięki czemu tabele pozostają spójne z wynikami programu.

8. Testowanie poprawności

Testy automatyczne sprawdzają zarówno wynik merytoryczny algorytmu, jak i elementy wymagane przez specyfikację. Zastosowano dziewięć testów obejmujących:

- dokładny zbiór 18 wzorców dla małego zbioru i sup=2;

- poprawne wsparcia i długości tidlist wybranych wzorców;
- rozróżnienie wzorców współwystępowania i następstwa, np. <(d f)> oraz <(d)(f)>;
- zgodność wyników dSPADE i bSPADE;
- zgodność wyników maxLength-dSPADE i maxLength-bSPADE;
- zgodność wariantu pełnego i ograniczonego przy wystarczająco dużym limicie;
- format i kompletność plików OUT i STAT;
- obsługę błędnych parametrów wejściowych;
- automatyczne rozpoznawanie i odczyt formatu SPMF.

Wynik testów: 9 testów zakończonych powodzeniem (9 passed).

9. Ograniczenia rozwiązania

Projekt spełnia wymagania funkcjonalne, jednak wyniki należy interpretować z uwzględnieniem następujących ograniczeń:

- Pomiary czasu wykonano jednokrotnie. W badaniu stricte wydajnościowym należałoby wykonać wiele powtórzeń i raportować medianę oraz rozrzut.
- Implementacja jest napisana w czystym Pythonie. Dla bardzo niskiego sup lub bardzo gęstych danych liczba kandydatów może rosnąć kombinatorycznie, co zwiększa czas i zużycie pamięci.
- Pozycje są przechowywane jako teksty i porządkowane leksykograficznie. Nie zmienia to semantyki itemsetów, lecz wpływa na ich kanoniczny zapis.
- Porównanie DFS i BFS dotyczy tej konkretnej implementacji. Inne struktury danych lub optymalizacje mogłyby zmienić relacje czasowe.
- Zbiory publiczne wykorzystano w formie udostępnionej przez SPMF. Raport opisuje statystyki obliczone przez program dla dostarczonych plików, a nie dla innych wersji tych baz.

10. Wnioski

Realizacja projektu potwierdziła przydatność reprezentacji pionowej w eksploracji wzorców sekwencyjnych. Po jednorazowym utworzeniu tidlist kolejne wsparcia mogą być wyznaczane przez operacje łączenia, bez ponownego skanowania całej bazy dla każdego kandydata.

Warianty dSPADE i bSPADE są równoważne pod względem odkrywanych wzorców. Różnią się kolejnością eksploracji i wymaganiami pamięciowymi: DFS koncentruje się na jednej gałęzi, natomiast BFS utrzymuje cały aktualny poziom. Wyniki eksperymentów nie wskazują jednej strategii zawsze szybszej; czas zależy od charakterystyki zbioru i liczby wzorców na poszczególnych poziomach.

Próg sup i limit maxLength są skutecznymi narzędziami kontroli przestrzeni poszukiwań. Wyższy sup zmniejsza liczbę częstych wzorców i kandydatów, natomiast maxLength pozwala świadomie pominąć wzorce dłuższe niż potrzebne w danym zastosowaniu.

Eksperymenty na BMSWebView1, SIGN i Kosarak 10k wykazały poprawne działanie programu dla baz różniących się liczbą sekwencji, długością sekwencji i rozmiarem dziedziny pozycji. Natywna obsługa formatu SPMF zwiększa odtwarzalność projektu i eliminuje konieczność ręcznej konwersji danych.

11. Bibliografia

1. M. J. Zaki, SPADE: An Efficient Algorithm for Mining Frequent Sequences, Machine Learning, 42, 2001, s. 31-60.

2. M. Kryszkiewicz, Wzorce sekwencyjne - materiały wykładowe, Politechnika Warszawska, 2021.

3. T. Morzy, Eksploracja danych: Metody i algorytmy, Wydawnictwo Naukowe PWN, 2013.

4. P. Fournier-Viger i in., SPMF: A Java Open-Source Pattern Mining Library, Journal of Machine Learning Research, 15, 2014.

5. SPMF - publiczne zbiory danych sekwencyjnych, repozytorium wykorzystane do pobrania BMSWebView1, SIGN i Kosarak 10k.

6. Python Software Foundation, Python 3 Documentation.